SQL · Tableau · Dashboard

Analyse der Pizzeria-Verkäufe

$817 860

Umsatz im Jahr 2015

Autor: Roman Luja · Data Analytics & Engineering · 06.10.2026

Die neue Inhaberin muss im Januar 3 Entscheidungen treffen

Laura Martínez hat eine Pizzeria in New Jersey übernommen – samt Kassendaten für das gesamte Jahr 2015.

1. Schichtplan

Keine unnötigen Mitarbeitenden in ruhigen Stunden und keine Überlastung zu Stoßzeiten.

2. Speisekarte

Zu viele Positionen, ein Teil der Zutaten verdirbt – was streichen?

3. Aktion

Wann und welche Aktion schwache Tage und Stunden belebt.

8 Kernfragen → Tableau-Dashboard → 3 Empfehlungen mit Zahlen.

Fazit: drei einfache Entscheidungen

1. Schichtplan

Mehr Personal mittags (12–14 Uhr) und abends (17–19 Uhr), weniger von 14 bis 16 Uhr.

2. Speisekarte

4 Pizzen 6 Wochen testweise streichen – sie bringen nur 6.9% des Umsatzes.

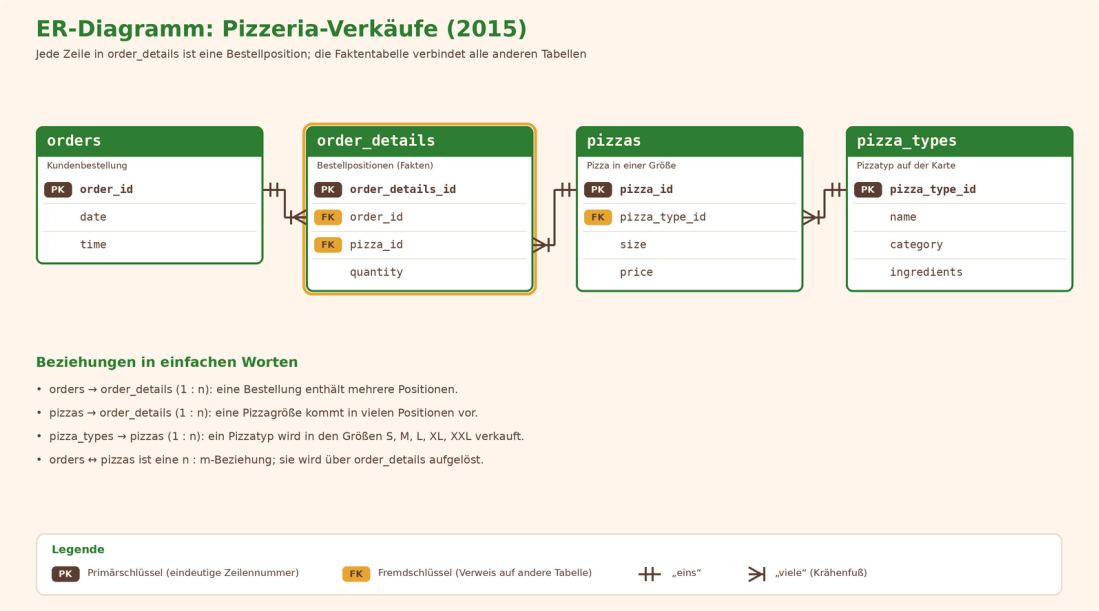
3. Aktion

Rabatt von Mo bis Mi, 14:00–16:00 Uhr, wenn wenig Gäste da sind.

Vier verknüpfte Tabellen, 48 620 Bestellpositionen

Tabelle	Zeilen
orders	21 350
order_details	48 620
pizzas	96
pizza_types	32

Quelle: Kaggle, Pizza Place Sales. Alle Tabellen werden per JOIN zur Auswahl sales_lines für Tableau verbunden.



Daten geprüft und analysebereit

- Saubere Daten: keine fehlenden Werte, keine Duplikate
- Tabellen ohne Verluste verbunden: 48 620 Zeilen = 48 620
- Umsatz = Preis × Menge
- Durchschnittlicher Bestellwert = Umsatz ÷ Anzahl der Bestellungen

```
-- Auswahl für Tableau: eine Zeile = eine Bestellposition
CREATE VIEW sales_lines AS
SELECT o.order_id, o.date AS order_date, o.time AS order_time,
       strftime('%m', o.date) AS month,
       CASE strftime('%w', o.date) WHEN '0' THEN 'So' WHEN '1' THEN 'Mo' WHEN
'2' THEN 'Di'
            WHEN '3' THEN 'Mi' WHEN '4' THEN 'Do' WHEN '5' THEN 'Fr' ELSE 'Sa'
END AS weekday,
       strftime('%H', o.time) AS hour,
       pt.name AS pizza_name, pt.category, p.size,
       CAST(p.price AS REAL) AS unit_price,
       CAST(od.quantity AS INTEGER) AS quantity,
       CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER) AS revenue
FROM order_details od
JOIN orders o       ON o.order_id = od.order_id
JOIN pizzas p       ON p.pizza_id = od.pizza_id
JOIN pizza_types pt ON pt.pizza_type_id = p.pizza_type_id;
```

Frage 1

Im Jahr hat die Pizzeria über 800 Tausend Dollar umgesetzt

$817 860

Umsatz im Jahr 2015

Das sind 21 350 Bestellungen. Ein Gast gibt im Schnitt $38 pro Besuch aus und nimmt 2.3 Pizzen.

SQL-Abfrage – im Anhang, Seite 21

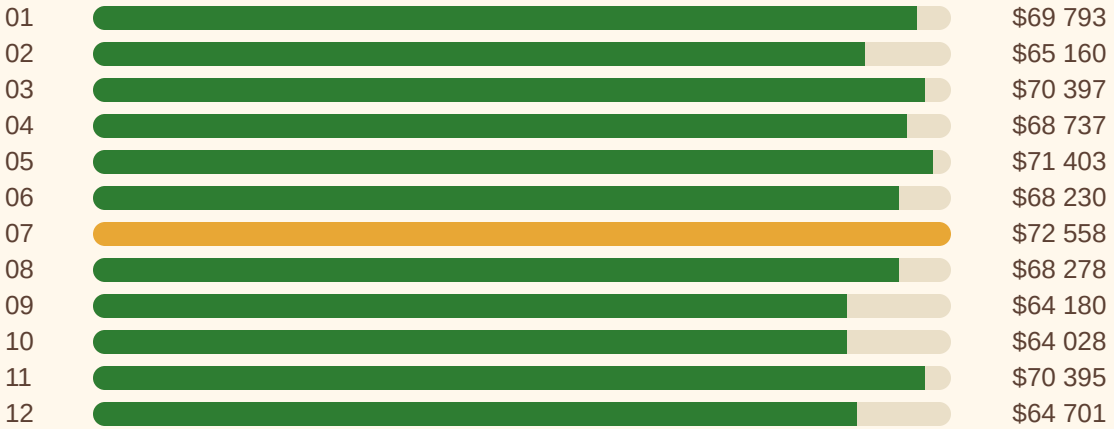
Frage 2

Juli ist der stärkste Monat, Oktober der schwächste

13%

Unterschied zwischen Juli und Oktober

Die Verkäufe sind das ganze Jahr über gleichmäßig: keine starken saisonalen Einbrüche.



SQL-Abfrage – im Anhang, Seite 21

Frage 3

Zwei Spitzen: mittags und abends, am stärksten ausgelastet ist der Freitag

45%

der Bestellungen kommen in 4 Stunden: 12–14 und 17–19 Uhr

Der Freitag ist am stärksten ausgelastet. Ruhig ist es von 14 bis 16 Uhr und spät abends.

	11	12	13	14	15	16	17	18	19	20	21	22	23
Mo	4.7	8.3	6.9	3.8	4.0	5.1	6.6	6.3	4.9	3.6	2.8	1.1	0.0
Di	3.6	8.1	7.1	3.8	3.7	5.7	5.9	5.8	5.3	4.1	2.5	1.6	0.0
Mi	4.4	8.1	7.3	3.9	4.0	5.0	6.6	6.4	4.6	3.7	2.7	1.4	0.0
Do	4.1	8.3	8.4	4.5	4.4	5.6	7.2	6.9	5.2	4.2	2.2	1.2	0.0
Fr	3.8	8.3	8.3	4.4	3.8	5.2	6.9	7.7	6.8	6.4	5.4	3.6	0.2
Sa	1.6	4.3	5.0	4.3	4.5	5.7	6.2	7.5	7.1	6.3	5.1	2.8	0.2
So	1.8	4.0	5.0	4.1	4.3	5.3	6.2	6.2	5.4	3.8	2.8	1.3	0.0

Durchschnittliche Bestellungen pro Stunde (Stunden 11–23, Zeilen = Wochentage). Je dunkler die Farbe, desto mehr Bestellungen

SQL-Abfrage – im Anhang, Seite 21

Frage 4

Den meisten Umsatz bringt Thai Chicken

№1

Thai Chicken beim Umsatz; Classic Deluxe wird am häufigsten verkauft

Drei Pizzen sind bei beiden Kennzahlen stark: Thai Chicken, Barbecue Chicken, Classic Deluxe. Am schwächsten ist Brie Carre.

SQL-Abfrage – im Anhang, Seite 22

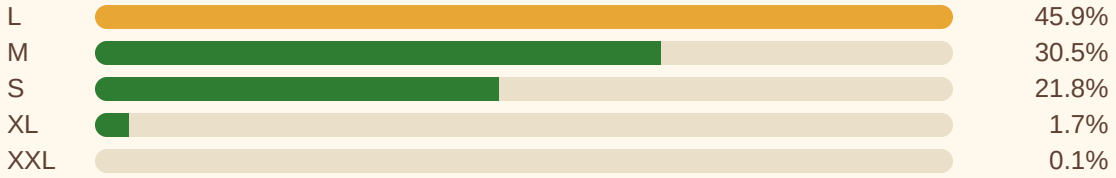
Frage 5

Das meiste Geld bringen große Pizzen (Größe L)

46%

des Umsatzes bringt Größe L

Alle Kategorien sind etwa gleich stark. Die Größen XL und XXL werden kaum gekauft.



SQL-Abfrage – im Anhang, Seite 22

Frage 6

4 Pizzen lassen sich ohne spürbare Verluste streichen

6.9%

des Umsatzes bringen diese 4 Pizzen ($56 182.5)

Brie Carre, Green Garden, Spinach Supreme, Mediterranean.
Empfehlung: Testzeitraum von 6 Wochen statt endgültiger Streichung.

SQL-Abfrage – im Anhang, Seite 23

Frage 7

Große Bestellungen sind selten, aber sehr wichtig

40%

des Umsatzes bringen Bestellungen ab 4 Pizzen

Nur jede fünfte Bestellung ist so groß. Für Firmenkunden lohnt sich ein eigenes Angebot.

SQL-Abfrage – im Anhang, Seite 23

Frage 8

Aktion: Montag bis Mittwoch, 14:00–16:00 Uhr

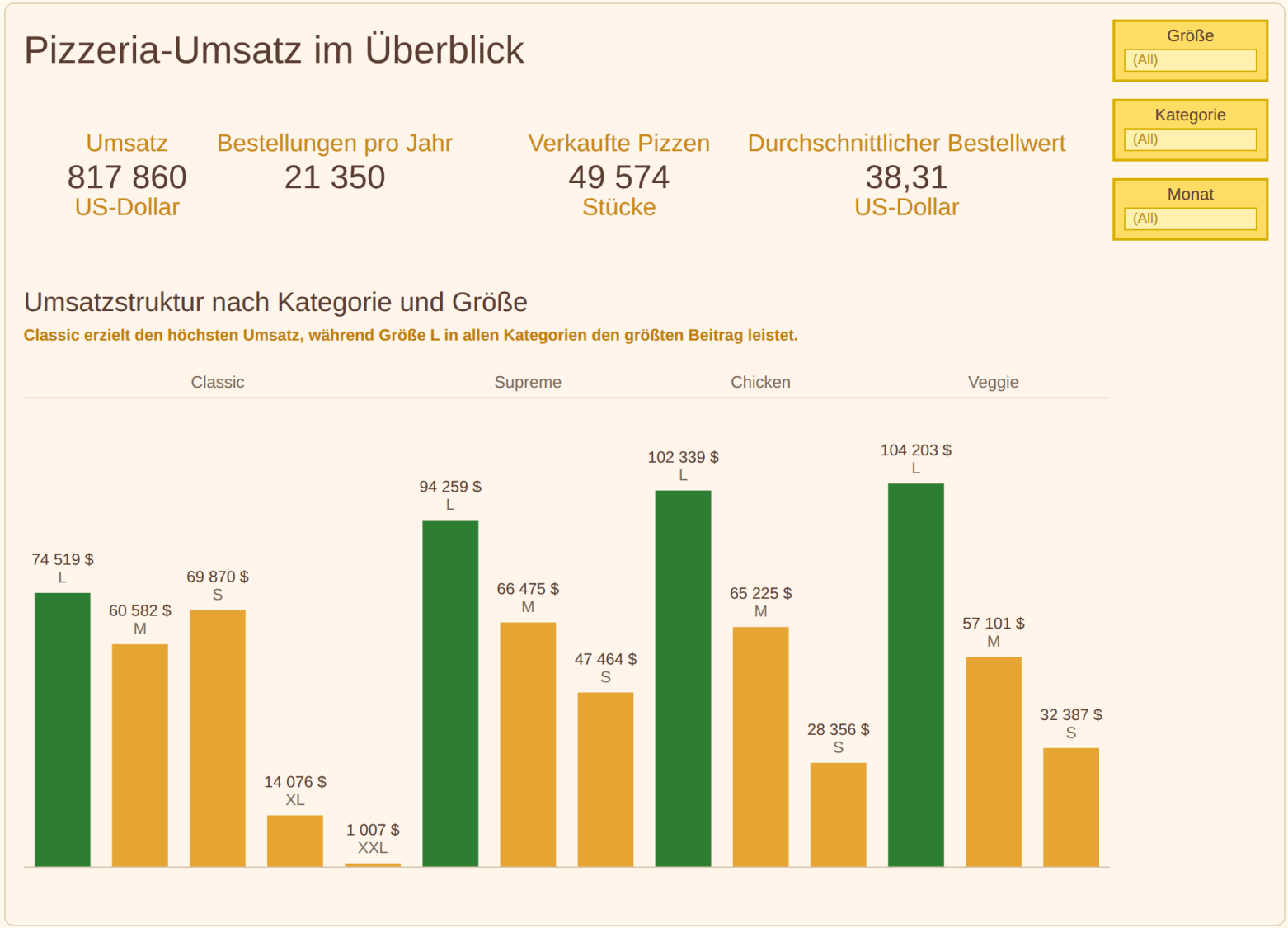
+$4.6 Tsd.

pro Jahr, wenn die Verkäufe in dieser Zeit um 10% steigen

Das ist die ruhige Zeit zwischen Mittag und Abend: derzeit rund $45.6 Tsd. Umsatz. Vorgeschlagen wird eine Aktion auf Classic-Pizzen in Größe L.

SQL-Abfrage – im Anhang, Seite 24

Interaktives Dashboard in Tableau (1/3)



Statische Kopie des Dashboards, aus den Daten nachgebaut. Interaktive Version mit Filtern: **auf Tableau Public öffnen**

Interaktives Dashboard in Tableau (2/3)

Zeitanalyse: Wann wird bestellt?

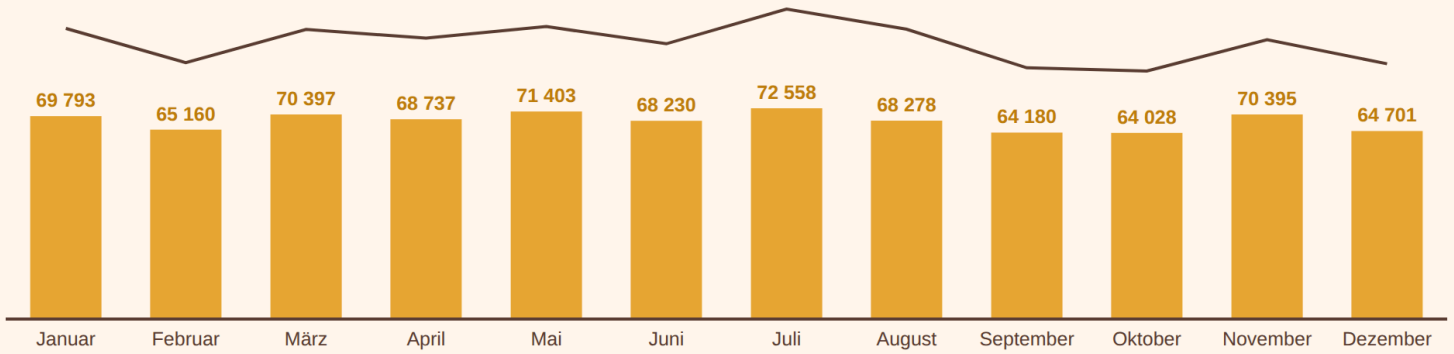
Monat: (All)

Größe: (All)

Kategorie: (All)

Umsatz und Bestellungen nach Monat

Balken zeigen den Umsatz, die Linie die Bestellungen. Höchststand im Juli, niedrigste Werte im September und Oktober.



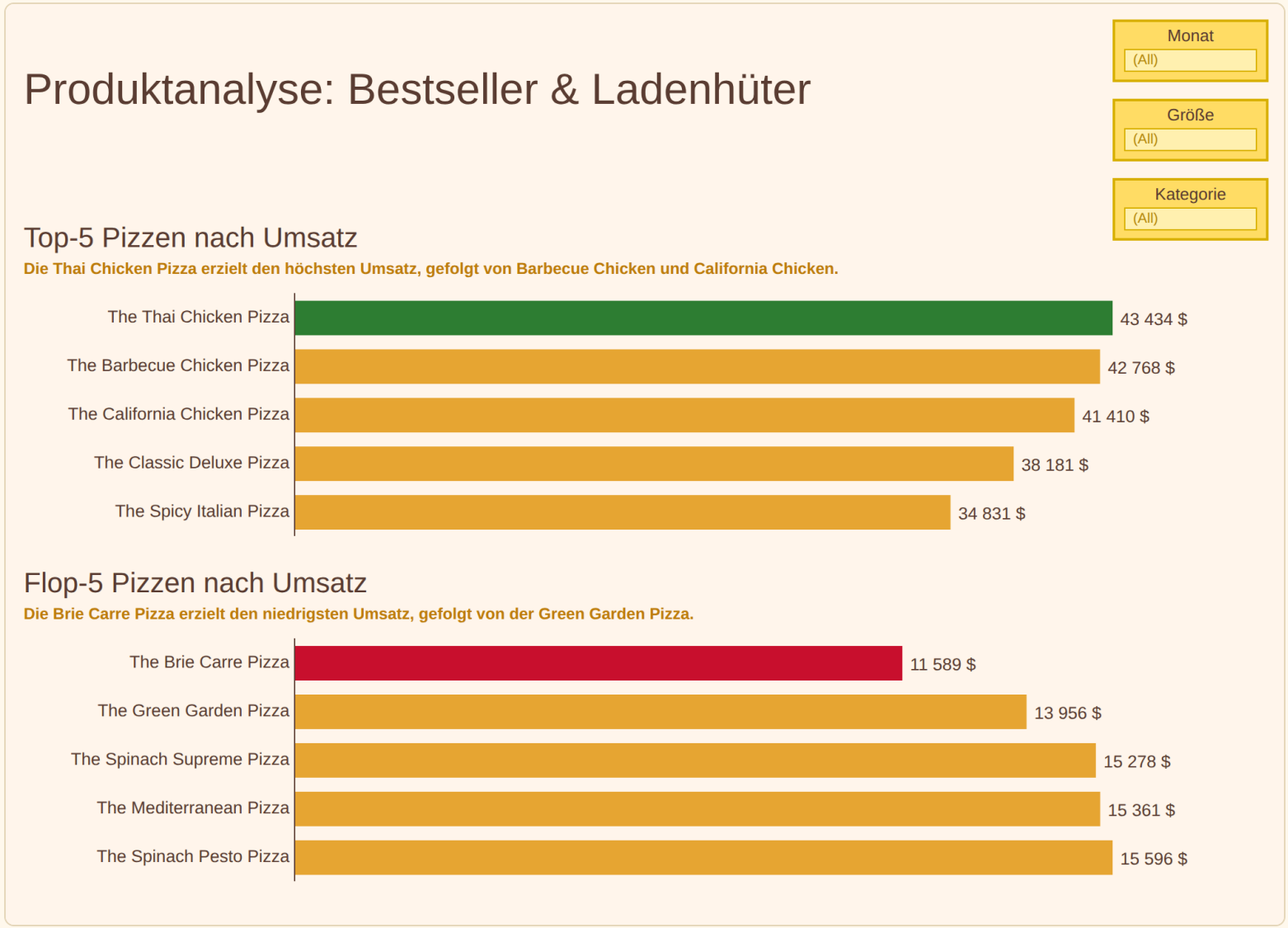
Bestellungen nach Wochentag und Stunde

Mo–Fr: Spitzen mittags (12–13 Uhr) und am frühen Abend (17–18 Uhr), dazwischen deutlich ruhiger; am Wochenende liegt die Spitze abends.

	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Montag		1	226	398	331	182	192	247	317	303	235	174	134	53	1
Dienstag	1		188	421	371	196	191	295	309	301	274	214	131	81	
Mittwoch		2	231	420	381	201	210	259	343	335	238	193	139	72	
Donnerstag		3	214	434	438	233	228	289	376	361	270	217	114	60	2
Freitag			192	415	413	221	190	258	344	387	340	319	268	180	11
Samstag		1	85	222	260	225	235	294	325	388	371	328	265	147	12
Sonntag		1	95	210	261	214	222	278	322	324	281	197	147	70	2

Statische Kopie des Dashboards, aus den Daten nachgebaut. Interaktive Version mit Filtern: **auf Tableau Public öffnen**

Interaktives Dashboard in Tableau (3/3)



Statische Kopie des Dashboards, aus den Daten nachgebaut. Interaktive Version mit Filtern: **auf Tableau Public öffnen**

Drei Kernergebnisse

Zwei Spitzen

Mittag und früher Abend: fast die Hälfte aller Bestellungen.

Ruhige Stunden

14 bis 16 Uhr und spät abends; der ruhigste Tag ist der Sonntag.

Schwache Pizzen

4 Positionen der Karte bringen nur 6.9% des Umsatzes.

Drei Empfehlungen für Laura

1. Schichtplan

WAS TUN

Volle Schicht mittags (12–14) und abends (17–19), Minimum von 14 bis 16 Uhr. Freitags und donnerstags die meisten Mitarbeitenden.

WARUM

Fast die Hälfte der Bestellungen kommt in diesen 4 Stunden.

2. Speisekarte

WAS TUN

Brie Carre, Green Garden, Spinach Supreme und Mediterranean für 6 Wochen testweise von der Karte nehmen, danach anhand des Ergebnisses entscheiden.

WARUM

Zusammen nur 6.9% des Umsatzes; ein Teil der Zutaten wird sonst nirgends gebraucht.

3. Aktion

WAS TUN

Mo–Mi von 14:00 bis 16:00 Uhr: Aktion auf Classic-Pizzen in Größe L. Für Firmen: ein Set mit Bonus.

WARUM

Ruhige Zeit; 10% Wachstum bringen rund $4.6 Tsd. pro Jahr (vor Kosten des Rabatts). Mit einem Test über 6 Wochen prüfen.

Das sollte man beachten

- Daten nur für ein Jahr (2015).
- Keine Kosten und Gehälter: Wir betrachten den Umsatz, nicht den Gewinn.
- Effekt der Aktion und der Streichung sind Schätzungen und müssen in der Praxis geprüft werden.
- Nächster Schritt: Aktion 6 Wochen testen.

Vielen Dank für Ihre Aufmerksamkeit!

Roman Luja · Data Analytics & Engineering · 06.10.2026

GitHub: **github.com/RomanLuja/pizza-sales-analysis**
Tableau Public: **public.tableau.com/…/Pizzeria-Umsatzimberblick**

Anhang: SQL-Abfragen (1/4)

Frage 1

```
SELECT
SUM(p.price * od.quantity) AS revenue,
COUNT(DISTINCT od.order_id) AS orders_cnt,
SUM(od.quantity) As pizzas_sold,
ROUND(SUM(p.price * od.quantity) / COUNT(DISTINCT order_id), 2) AS avg_check,
ROUND(CAST(SUM(od.quantity) AS REAL) / COUNT(DISTINCT od.order_id), 1)  AS avg_pizzas_per_order
FROM pizzas AS p
JOIN order_details AS od
        ON p.pizza_id = od.pizza_id;
```

Frage 2

```
WITH monthly AS (
    SELECT
        strftime('%m', o.date) AS month,
        SUM(p.price * od.quantity)              AS revenue,
        COUNT(DISTINCT od.order_id)             AS orders_cnt
    FROM orders AS o
    JOIN order_details AS od USING (order_id)
    JOIN pizzas AS p USING (pizza_id)
    GROUP BY month
)
SELECT
    month,
    revenue,
    orders_cnt,
    ROUND(
        100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
              / LAG(revenue) OVER (ORDER BY month),
        1
    ) AS revenue_change_pct,
    CASE
        WHEN revenue = (SELECT MAX(revenue) FROM monthly) THEN 'stärkster Monat'
        WHEN revenue = (SELECT MIN(revenue) FROM monthly) THEN 'schwächster Monat'
        ELSE ' '
    END AS note
FROM monthly
ORDER BY month;
```

Frage 3

```
-- Anzahl der Bestellungen für jeden Kalendertag
WITH daily_orders AS (
    SELECT
        o.date,
        strftime('%w', o.date) AS weekday_num,
        COUNT(DISTINCT o.order_id) AS orders_cnt
    FROM orders AS o
    GROUP BY o.date
)
SELECT
    CASE weekday_num
        WHEN '0' THEN 'Sonntag'
        WHEN '1' THEN 'Montag'
        WHEN '2' THEN 'Dienstag'
        WHEN '3' THEN 'Mittwoch'
        WHEN '4' THEN 'Donnerstag'
        WHEN '5' THEN 'Freitag'
        WHEN '6' THEN 'Samstag'
    END AS weekday,
    ROUND(AVG(orders_cnt), 1) AS avg_orders
FROM daily_orders
GROUP BY weekday_num
ORDER BY weekday_num;

-- Anzahl der Bestellungen je Stunde (gesamtes Jahr)
SELECT
    strftime('%H', o.time) AS hour,
    COUNT(DISTINCT o.order_id) AS orders_cnt
FROM orders AS o
GROUP BY strftime('%H', o.time)
ORDER BY orders_cnt DESC;
```

Anhang: SQL-Abfragen (2/4)

Frage 4

```
WITH pizza_sales AS (
    SELECT
        pt.name AS pizza_name,
        SUM(p.price * od.quantity) AS revenue,
        SUM(od.quantity) AS pizzas_sold
    FROM order_details AS od
    JOIN pizzas AS p
        ON od.pizza_id = p.pizza_id
    JOIN pizza_types AS pt
        ON p.pizza_type_id = pt.pizza_type_id
    GROUP BY pt.name
)

SELECT * FROM (
    SELECT 'Top 5 Umsatz' AS ranking, pizza_name, revenue, pizzas_sold
    FROM pizza_sales
    ORDER BY revenue DESC
    LIMIT 5
)

UNION ALL

SELECT * FROM (
    SELECT 'Letzte 5 Umsatz' AS ranking, pizza_name, revenue, pizzas_sold
    FROM pizza_sales
    ORDER BY revenue ASC
    LIMIT 5
)

UNION ALL

SELECT * FROM (
    SELECT 'Top 5 Menge' AS ranking, pizza_name, revenue, pizzas_sold
    FROM pizza_sales
    ORDER BY pizzas_sold DESC
    LIMIT 5
)

UNION ALL

SELECT * FROM (
    SELECT 'Letzte 5 Menge' AS ranking, pizza_name, revenue, pizzas_sold
    FROM pizza_sales
    ORDER BY pizzas_sold ASC
    LIMIT 5
);
```

Frage 5

```
-- Umsatzanteil nach Kategorien

SELECT
    pt.category,
    SUM(p.price * od.quantity) AS revenue,
    ROUND(
        100.0 * SUM(p.price * od.quantity)
        / SUM(SUM(p.price * od.quantity)) OVER (),
        1
    ) AS revenue_share_pct
FROM order_details AS od
JOIN pizzas AS p
    ON od.pizza_id = p.pizza_id
JOIN pizza_types AS pt
    ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY revenue DESC;

-- Umsatzanteil nach Pizzagröße

SELECT
    p.size,
    SUM(p.price * od.quantity) AS revenue,
    ROUND(
        100.0 * SUM(p.price * od.quantity)
        / SUM(SUM(p.price * od.quantity)) OVER (),
        1
    ) AS revenue_share_pct
FROM order_details AS od
JOIN pizzas AS p
    ON od.pizza_id = p.pizza_id
GROUP BY p.size
ORDER BY revenue DESC;
```

Anhang: SQL-Abfragen (3/4)

Frage 6

```
-- Umsatz und Verkäufe je Pizzasorte (alle Größen zusammen)
WITH pizza_sales AS (
    SELECT
        pt.name                    AS pizza_name,
        SUM(p.price * od.quantity) AS revenue,
        SUM(od.quantity)           AS pizzas_sold
    FROM order_details AS od
    JOIN pizzas AS p
        ON od.pizza_id = p.pizza_id
    JOIN pizza_types AS pt
        ON p.pizza_type_id = pt.pizza_type_id
    GROUP BY pt.pizza_type_id, pt.name
    HAVING SUM(od.quantity) > 0              -- nur Sorten mit Verkäufen
),

-- Umsatzanteil, kumulierter Anteil und Rang (aufsteigend nach Umsatz)
ranked AS (
    SELECT
        pizza_name,
        revenue,
        pizzas_sold,
        ROUND(100.0 * revenue / SUM(revenue) OVER (), 2) AS revenue_share_pct,
        ROUND(
            100.0 * SUM(revenue) OVER (
                ORDER BY revenue ASC, pizza_name
                ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
            ) / SUM(revenue) OVER (),
            2
        ) AS cumulative_share_pct,
        ROW_NUMBER() OVER (ORDER BY revenue ASC, pizza_name) AS rn_revenue
    FROM pizza_sales
),

-- Entscheidung: die 4 schwächsten Sorten werden getestet
candidates AS (
    SELECT
        pizza_name, revenue, pizzas_sold, revenue_share_pct, cumulative_share_pct,
        CASE WHEN rn_revenue <= 4 THEN 'Testkandidat' ELSE 'behalten' END AS decision
    FROM ranked
    WHERE rn_revenue <= 5                    -- 5. Zeile zum Vergleich
)

SELECT
    pizza_name, revenue, pizzas_sold, revenue_share_pct, cumulative_share_pct, decision,
    ROUND(SUM(revenue) OVER (PARTITION BY decision), 2) AS revenue_per_decision
FROM candidates
ORDER BY revenue ASC;
```

Frage 7

```
-- Pizzen und Umsatz je Bestellung
WITH order_sales AS (
    SELECT
        od.order_id,
        SUM(od.quantity)           AS pizzas_count,
        SUM(p.price * od.quantity) AS revenue
    FROM order_details AS od
    JOIN pizzas AS p
        ON od.pizza_id = p.pizza_id
    GROUP BY od.order_id
)

-- Gruppierung: 4+ Pizzen gegenüber 1–3 Pizzen
SELECT
    CASE
        WHEN pizzas_count >= 4 THEN '4+ Pizzen'
        ELSE '1–3 Pizzen'
    END AS order_group,
    COUNT(*) AS orders_cnt,
    ROUND(SUM(revenue), 2) AS revenue,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS orders_share_pct,
    ROUND(100.0 * SUM(revenue) / SUM(SUM(revenue)) OVER (), 1) AS revenue_share_pct,
    ROUND(SUM(revenue) / COUNT(*), 1) AS avg_order_value
FROM order_sales
GROUP BY
    CASE
        WHEN pizzas_count >= 4 THEN '4+ Pizzen'
        ELSE '1–3 Pizzen'
    END
ORDER BY order_group;
```

Anhang: SQL-Abfragen (4/4)

Frage 8

```
-- Basis für die Wahl des Zeitfensters (Heatmap Wochentag × Uhrzeit): Bestellungen um 14:00 und 15:00 Uhr
-- je Wochentag, gesamt und pro Tag
SELECT
    CASE strftime('%w', o.date)
        WHEN '0' THEN 'Sonntag'
        WHEN '1' THEN 'Montag'
        WHEN '2' THEN 'Dienstag'
        WHEN '3' THEN 'Mittwoch'
        WHEN '4' THEN 'Donnerstag'
        WHEN '5' THEN 'Freitag'
        WHEN '6' THEN 'Samstag'
    END AS weekday,
    COUNT(DISTINCT o.order_id) AS orders_cnt,
    COUNT(DISTINCT o.date)     AS days_cnt,
    ROUND(1.0 * COUNT(DISTINCT o.order_id) / COUNT(DISTINCT o.date), 1) AS avg_orders_per_day
FROM orders AS o
WHERE strftime('%H', o.time) IN ('14','15')          -- 14:00-16:00 Uhr
GROUP BY strftime('%w', o.date)
ORDER BY strftime('%w', o.date);

-- Aktionspotenzial: Montag bis Mittwoch, 14:00-16:00 Uhr, Aktionsprodukt Classic L (Aufgabe 3 und 5)
SELECT
    'Montag–Mittwoch' AS weekday,
    '14:00–16:00'     AS hour,
    COUNT(DISTINCT o.order_id) AS orders_cnt,

    -- Umsatz des gesamten Zeitfensters
    ROUND(SUM(od.quantity * p.price), 2)        AS current_revenue,
    ROUND(SUM(od.quantity * p.price) * 1.10, 2) AS revenue_after_10_pct,
    ROUND(SUM(od.quantity * p.price) * 0.10, 2) AS additional_revenue,

    -- Umsatz des Aktionsprodukts Classic L im selben Fenster
    ROUND(SUM(CASE WHEN pt.category = 'Classic' AND p.size = 'L'
                   THEN od.quantity * p.price END), 2) AS classic_l_revenue,
    ROUND(100.0 * SUM(CASE WHEN pt.category = 'Classic' AND p.size = 'L'
                           THEN od.quantity * p.price END)
          / SUM(od.quantity * p.price), 1)              AS classic_l_share_pct

FROM orders AS o
JOIN order_details AS od ON o.order_id = od.order_id
JOIN pizzas AS p ON od.pizza_id = p.pizza_id
JOIN pizza_types AS pt ON p.pizza_type_id = pt.pizza_type_id

WHERE strftime('%w', o.date) IN ('1','2','3')        -- Montag bis Mittwoch
  AND strftime('%H', o.time) IN ('14','15');         -- 14:00-16:00 Uhr
```